Samir Rathore

Design Document

I. Introduction

a. My project is going to be object detection and tracking in real time. I am interested in this project because in one of my other classes we were learning about AI voice and facial feature cloning. Object detection and tracking is necessary for artificial intelligence to see their environments through detection, which I find interesting. Taking on this project allows me to learn how object detection works and is built. YOLOv8 is a model I will be using to compare against my Object Detection through OpenCV.

b. Before tackling this project there was a lot of research I had to do. I had to learn what libraries I wanted to use, what approach I wanted to take, and overall understand what my code needed to include. In my research I came across many different libraries that could help me with my project. OpenCV and YOLO were the libraries that kept popping up for me, and for good reason. After playing around with YOLOv8, I found that it is incredible software and is so easy to use. YOLOv8 is an object detection model developed by Ultralytics which is a faster and more accurate version of the YOLOv5 model which was industry-defining for object detection and tracking. There are several different pretrained models for YOLOv8 that all differ in their speed, FLOPs, and more.

Model	size (pixels)	mAP^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Fig.1: YOLOv8 Image from GitHub

As you can see, YOLOv8n has the fastest speed and since my computer doesn't have a GPU and I'm aiming to maximize my fps, I will use v8n. I tried to run a simple webcam identification on any other model, but every version but the nano version gave me extremely low fps. I found that to run the higher versions such as v8l and v8x, you need a powerful GPU just to achieve 50 frames per second. I will compare YOLOv8n against my OpenCV implementation of Object Detection and observe the confidence factors of both to determine which is more accurate.

c. In terms of what code is available on the internet, I linked the websites that I used to help write my code. I also searched up the documentation for OpenCV and Matplotlib commands that I needed that I wasn't familiar with such as FuncAnimation from Matplotlib. For OpenCV, since I was using a pretrained model from SSD Mobilenet, I had to search up documentation on how to use weights and config to pass into the Detection Model command. Many people have done OpenCV implementation of YOLOv8 before, but I did it differently. Most of the similar projects used OpenCV to create bounding boxes but still used the

YOLOv8 model to make predictions. My goal was to compare the YOLOv8 model and OpenCV model and display them side by side and compare them through their confidence values and accuracy.

II. Design Process

a. The first thing I did was start off by downloading the Ultralytics library. I then wrote a few lines of code to just get a feel of YOLOv8 and test out the different models that are available. After realizing how easy it is to use the pretrained prediction model, I had to figure out a way to make this project harder. I think it would be perfect for this project to create my own custom predication model. This is where my project took a massive turn. Originally, I was planning on training YOLOv8 on a custom dataset to improve the accuracy of cars, motorcycles, and bicycles. I used RoboFlow to download annotated images of cars and motorcycles which provides a .yaml file. This .yaml file along with the annotated images is passed through the model.train() function and is used to train YOLOv8n. However, I found that my laptop was running into issues with how Ultralytics was installed, and I spent multiple days trying to find fixes online. A lot of the projects used with YOLOv8n are done in Google Colab, something I am very unfamiliar with, so it was hard to try to find fixes and then implement them in VS Code. It eventually came to a point where I had to think about changing my project, so I decided I would try to implement Object Detection through OpenCV and compare it with YOLOv8n predictions. This would also allow me to use many of the things we learned in class for my project.

b. Once I determined that I would compare OpenCV to YOlOv8n, I had to start writing the code for cv2 detection. This led me to searching up different models online and I came across the coco_2020 dataset and the frozen inference graph. I tried it and since it worked, I continued with this model.

```
#OpenCV model ---- mobileNet SSD Method
configPath ='ssd_mobilenet_v3_large_coco_2020_01_14.pbtxt'
weighsPath ='frozen_inference_graph.pb'
cvmodel = cv2.dnn_DetectionModel(weighsPath, configPath)
cvmodel.setInputSize(width,height)                          #Sets limit of input size (pixels, pixels)
cvmodel.setInputScale(1.0/127.5)                            #Normalize pixels
cvmodel.setInputMean((127.5,127,5,127.5))
cvmodel.setInputSwapRB(True)                                #BGR to RGB
```

Fig.2: OpenCV model

c. The two biggest hurdles I encountered were creating the codec and creating the bounding boxes for the OpenCV detection model. For some reason, when I tried to write the output file in a 'XVID' format, the output video was only receiving one frame. At first, I thought it was with how I was outputting each frame to the out file but after time I realized everything was right. That was when I changed the codec to an mp4 format using FMP4 but kept the output file as output_file.avi. This worked and I kept running with .avi since the prediction file created by YOLOv8n was also a .avi file.

d. The bounding boxes was difficult for me since I wasn't familiar with how I wanted to go about implementing this. After a lot of iterations, I got a somewhat consistent output of correct bounding boxes, with correct labels and colors, and decided that it was good enough for comparing the two models.

```
#Creates boxes and labels each object identified, the same class will get same color (similar to YOLOv8)
if(len(ClassIndex != 0)):
    for ClassInd, conf, boxes in zip(ClassIndex.flatten(), confidence.flatten(), bbox):
        if(ClassInd <= 80):

            #Use the unique color associated with class index
            color = class_colors[ClassInd]
            cv2.rectangle(frame, boxes ,color ,2)

            #Add class name and confidence to label
            label = f"{names_list[ClassInd - 1]}: {conf:.2f}"
            cv2.putText(frame, label, (boxes[0]+10, boxes[1]+40), fontFace = cv2.FONT_HERSHEY_PLAIN, fontScale = 1, color = color, thickness = 2)
```

Fig.3: Bounding Boxes

This is the code for implementing the bounding boxes. The first line is to ensure that that there are detected objects in the frame before proceeding with the rest of the code. Before I implemented this line, I was getting crazy bounding boxes on random items, this step was crucial for the rest of the code. For the next line, I was trying to get the information out of the detected class and put it into the arrays listed. I flattened the arrays because the arrays were multi-dimensional NumPy arrays, flattening them allowed me to iterate over the elements of these arrays in a single loop. The if statement following the for loop just checks if the number of classes is less than the maximum provided class list which is in the names.txt file. I then follow with assigning colors to each bounding box based on what class it is, the code for class_colors is above this code in a different section. The next line draws the bounding box in a rectangle shape, like YOLOv8n, and uses the coordinates I specified in the boxes array, the thickness 2 is the number of pixels. Finding what coordinates to use was very difficult, I was playing around with them and after a while, it worked consistently. The next line creates the label for the object which includes the name and the confidence score. The next line physically displays the parameters I mentioned above onto the frame, along with several cosmetic adjustments to make the bounding boxes look better.

e. After both the YOLOv8n and OpenCV prediction videos were created, the next

part came to display both side by side. I started off by importing matplotlib and several classes from it. I then had to create two VideoCapture objects for each video, and make sure that both opened correctly. If this doesn't work, double check that you followed the steps in the documentation file correctly to determine the file path in PART I, section c. I then create the figure and the subplots that each of the videos will go on.

f. To update each frame which I will pass into FuncAnimation, I created a function called update. It starts by reading the frames of the OpenCV video and the YOLOv8n video and stops them reading when the frames are finished. The setup blocks are used to change the color from BGR to RGB. I implemented this process because if you don't do this, you will get weird colors in the video, almost as if a colorblind filter was applied. Since imshow() assumes a numpy array, I had to first convert it to a numpy array and then swap the color channels. I also added the figure titles for each subplot in this step.

g. I then calculated the total frames present in OpenCV's video. The reason I use cap1 and not cap2 is because of the feature I implemented where you can close the video early using the spacebar. If you end it early, it will cause the number of frames of each of the videos to be different, which causes the code not to compile correctly once the OpenCV video runs out of frames to play. This is a safety feature where the frames are limited to the minimum number of frames between both videos.

h. Then I run the command which creates the actual animation. It passes the matplotlib figure which contains the subplots, the update function, and number of frames you want it to run, the interval in milliseconds, and if you want it to repeat

once over. The plt,show() shows the figure and then I release the resources so that the memory doesn't run out and cause you to blue screen.

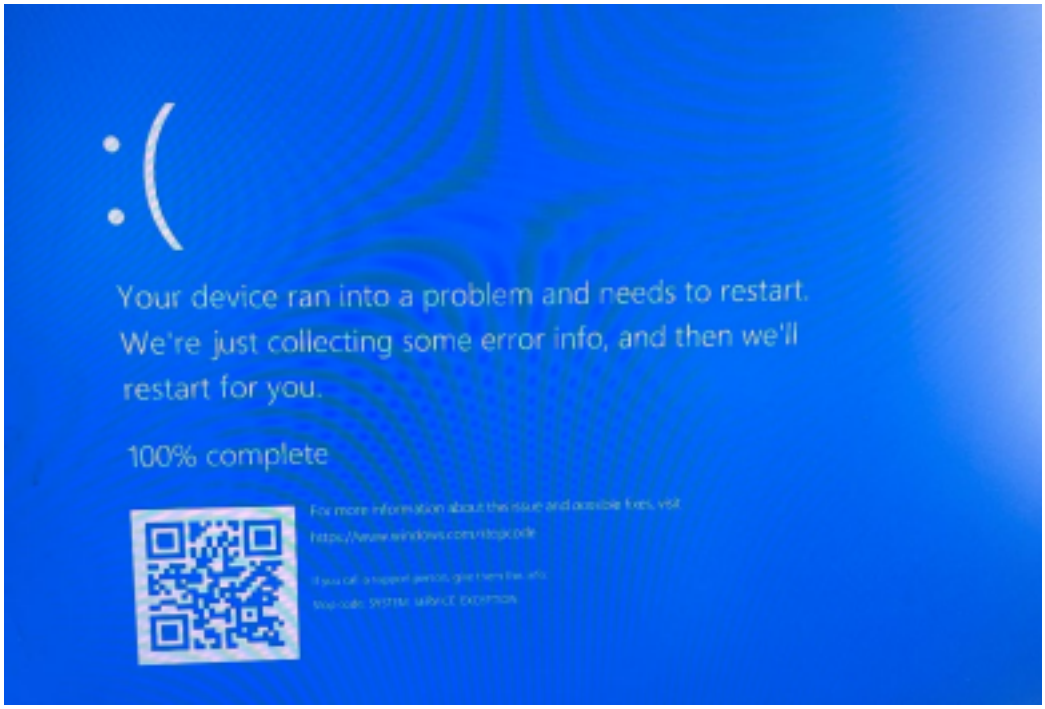

Fig.4: What happens when you don't have a GPU or a lot of memory.

III. Results

a. The are 3 main results that you will get when you run the file v8_vs_cv.py. The first is a test_video.avi file which is produced from running the YOLOv8n model on the video test_video.mp4 along with a window popup of the results in real time. The second is OpenCV's prediction model on the same video displayed on a window popup which also occurs in real-time, though slower, this video will be saved to a file called output_video.avi. The last is these two videos side by side. One thing to note is that the output_video.avi will only be as long as you leave it. Meaning, since OpenCV takes a lot longer to run through each frame, it appears choppy on the display, so I added a quit command that closes the popup and continues to the side-by-side comparison. However, if you press it early, the

number of frames which will save to the output_video.avi will be very short and therefore the comparison video will also be short. When running, try to let the window popup run for a bit before closing the window using the spacebar. Here are some of the things you will see after you run the code.

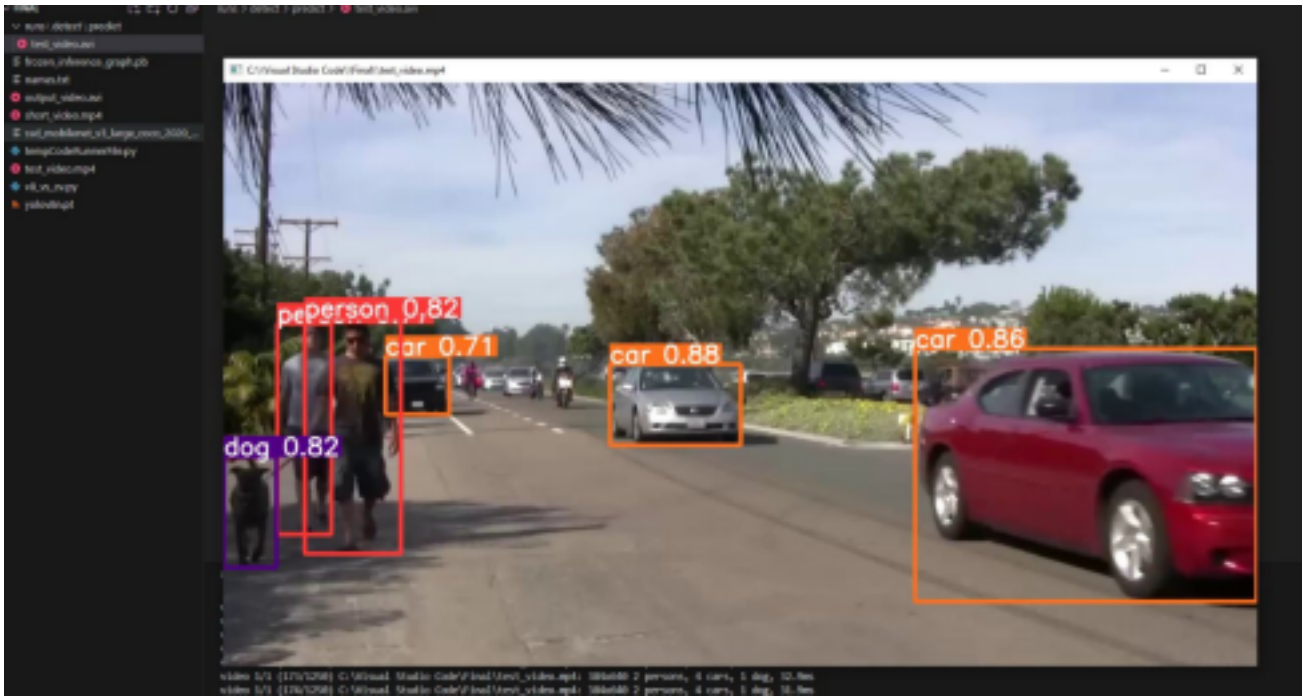

Fig.5: When running the code, this folder will first popup with the test_video.avi file which contains the prediction made by YOLOv8n. A window popup with real-time predictions will also be displayed

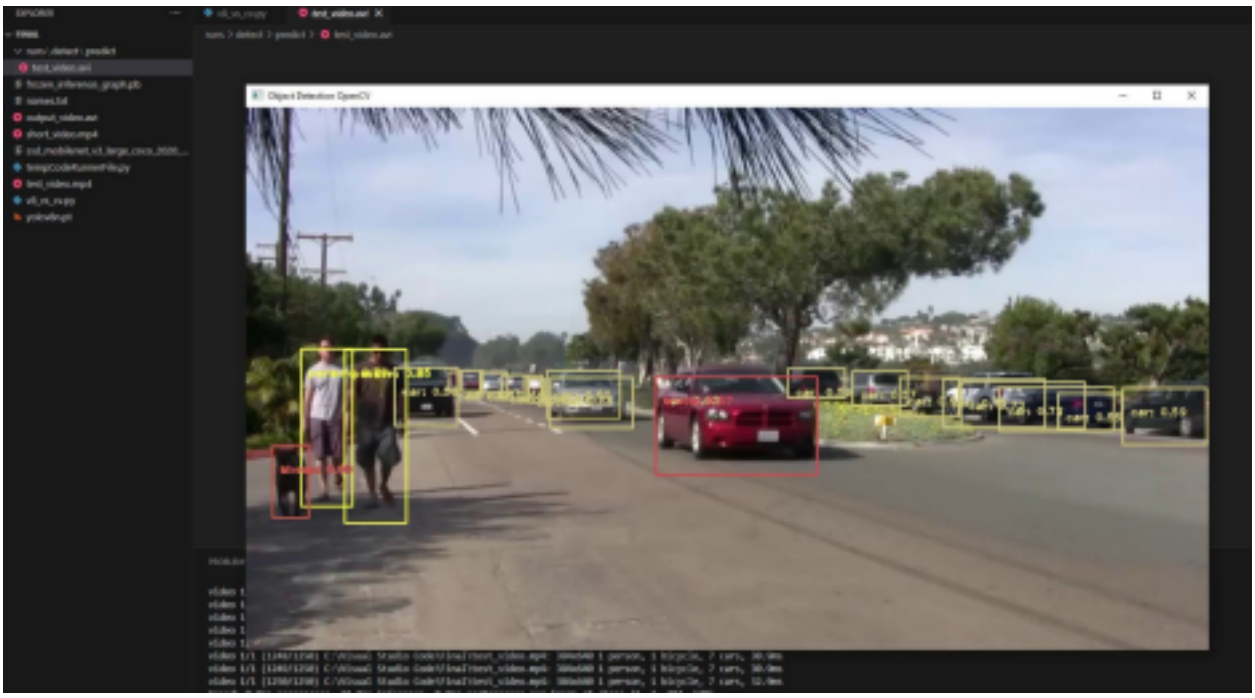

Fig.6: After the YOLOv8n predictions are complete, OpenCV will make predictions frame by frame in a window popup that will look something like this and save them to output_file.avi.

Fig.7: Once you either wait for the previous window to close, or close it manually using the spacebar, this window will popup. I made the dimensions smaller so that it runs somewhat smoothly if you don't have a GPU on your computer. This should run faster than the last popup since it is using FuncAnimation.

As we can see, YOLOv8 does a much better job at Object Detection and Tracking than OpenCV's model. The confidence scores are higher, and the class identification is more accurate. With the image above you can see that OpenCV believes the dog to be a person while YOLOv8 correctly identifies this. There are more instances where OpenCV predicts the wrong class, however it still does identify and object, where sometimes YOLOv8 does not. This might be because I set a limit to how low the confidence scores can go, however a confidence score of less than 0.5 is, in my opinion, not good enough.

IV. Concepts Learned

a. YOLOv8n

i. One of the biggest things I learned about through this project was the groundbreaking open source SOTA model built by Ultralytics, YOLOv8. I was very unaware of how Object Detection worked before I started this project, and I am glad that I learned more about it as well as how to implement it in python. Though my code is short for the YOLOv8n model, all thanks to the model command, there was a lot I had to learn before this. I originally started off with passing my webcam into the arguments of the model, which allowed me to detect objects around me. I then fell down the rabbit hole of trying to train YOLOv8n using a coco128 dataset which worked, but when I tried to train it using my own annotated data, it didn't, which is when I decided to shift my project to more OpenCV based. However, the process of trying to train YOLOv8n taught me about the segmentation, prediction, and training commands that YOLO offers.

b. OpenCV

i. I was familiar with OpenCV due to the Lab 5 assignment we had during the semester. Reading and writing files were all things familiar to me, therefore I used some of the ideas in my code. While we worked with images in that homework assignment, it was a little different using video files. I had to learn about the VideoWriter class and VideoCapture class to incorporate videos into cv2. I also had to learn how to use .rectangle() and .putText() from cv2, since I didn't know how to pass arguments correctly into them. There were also a variety of other commands which I learned which helped with overall performance of displaying the videos such as .release(), .destroyAllWindows(), and .waitKey().

c. Matplotlib

i. Although familiar with using Matplotlib and making subplots to display different images side-by-side, I needed a different class from Matplotlib called FuncAnimation. This class allowed me to create the animation of the YOLOv8 video and OpenCV video by passing each frame into an update function. It is a useful command and is faster than cv2.imShow(), which I was previously using before I learned about this.

d. Random

i. Everything I used from the random module I learned through class. I used this module to generate random colors in BGR format for the bounding boxes in the OpenCV model.

V. Retrospective

a. There are several things I would have liked to do if I had more time. The biggest thing I wanted to do was originally sit by Route 1 with a webcam and try both models in actual real-time. While performing it on a video is the same in essence, I think it would be cooler to try it in that type of environment. I would have also liked to train the YOLOv8 model with my own data. I spent a lot of time annotating specific images of motorcycles, cars, and bicycles, but I didn't end up using them. I could have improved the accuracy of YOLOv8, showing just how much better it is than OpenCV's model detection. I had fun with this project, and it taught me a lot about how Object Detection works.

Sources

chr043416@gmail.com, et al. "Python Usage." *Python - Ultralytics YOLOv8 Docs*, Ultralytics, 12 Nov. 2023, docs.ultralytics.com/usage/python/.

Hunter, John, et al. "Animations Using Matplotlib#." *Animations Using Matplotlib - Matplotlib 3.8.2 Documentation*, Matplotlib, matplotlib.org/stable/users/explain/animations/animations.html. Accessed 11 Dec. 2023.

OnlyHDVideos, director. *30 Minutes of Cars Driving By in 2009*. *YouTube*, YouTube, 12 Aug. 2010, https://www.youtube.com/watch?v=e_WBuBqS9h8. Accessed 11 Dec. 2023.

Ostos, Edgar Florez. "MobileNet SSD Object Detection Opencv 3.4.1 DNN Module." *Ebenezer Technologies - Artificial Intelligence*, Ebenezer Techs, 29 Aug. 2022, ebenezertechs.com/mobilenet-ssd-using-opencv-3-4-1-deep-learning-module-python/.

R.ZaneR.Zane, et al. "OpenCV Videowriter Not Writing to Output.Avi." *Stack Overflow*, 1 Apr.

1965, stackoverflow.com/questions/54738747/opencv-videowriter-not-writing-to-output avi.

Sidpro-Hash. “Sidpro-Hash/Object-Detection: Object Detection Using SSD Mobile Net V3.” *GitHub*, 4 Oct. 2021, github.com/sidpro-hash/Object-Detection.

Syed Saad, and ZdaRZdaR. “OpenCV from Python Shows Different Results for JPG and PNG Images?” *Stack Overflow*, 1 Feb. 1966, stackoverflow.com/questions/59333332/opencv from-python-shows-different-results-for-jpg-and-png-images.